RAJALAKSHMI ENGINEERING COLLEGE

ous Institution, Affiliated to Anna University, Chennai)

(An Autonomous Institution, Affiliated to Anna University, Chennai)

SENTIMENT ANALYSIS USING RECURRENT NEURAL NETWORKS

Submitted by

S. JEENESHWAAR (241501073)

AI23531 DEEP LEARNING

Department of Artificial Intelligence and Machine Learning

Rajalakshmi Engineering College,
Rajalakshmi Nagar, Thandalam
Tamil Nadu – 602 105

OCTOBER 2026

BONAFIDE CERTIFICATE

NAME	S. JEENESHWAAR
ACADEMIC YEAR	2026 – 2027
SEMESTER	V
BRANCH	Artificial Intelligence and Machine Learning
UNIVERSITY REGISTER NO.	241501073

Certified that this is the bonafide record of work done by the above student in the Mini Project titled “Sentiment Analysis Using Recurrent Neural Networks” in the subject AI23531 Deep Learning during the year 2026–2027.

Signature of Faculty – in – Charge

Submitted for the Practical Examination held on _____

INTERNAL EXAMINER EXTERNAL EXAMINER

ABSTRACT

Sentiment analysis is a natural language processing task that identifies the opinion expressed in a piece of text. This project develops a deep learning model to classify movie reviews as positive or negative using the Internet Movie Database (IMDB) movie-review dataset available through TensorFlow Keras. The model represents reviews as integer sequences using a vocabulary limited to 10,000 words and pads or truncates each sequence to 200 tokens. The neural network consists of an embedding layer, a Long Short-Term Memory (LSTM) layer with 64 units, a dropout layer, and a sigmoid output layer. The model was trained for three epochs with a batch size of 64. Training accuracy reached 91.83%, while validation accuracy reached 86.58% in the recorded run. The separate test-set evaluation and validated interactive prediction function should be added after those implementation steps are completed. This project demonstrates the use of sequential deep learning for automated opinion classification.

Keywords: Sentiment Analysis, Deep Learning, Long Short-Term Memory, Recurrent Neural Network, Natural Language Processing, IMDB Dataset, TensorFlow, Keras.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	III
1	INTRODUCTION	1
2	LITERATURE REVIEW	3
3	SYSTEM REQUIREMENTS	4
3.1	HARDWARE REQUIREMENTS	4
3.2	SOFTWARE REQUIREMENTS	4
3.3	LIBRARIES USED	4
4	SYSTEM OVERVIEW	5
4.1	EXISTING SYSTEM	5
4.1.1	DRAWBACKS OF EXISTING SYSTEM	5
4.2	PROPOSED SYSTEM	5
4.2.1	ADVANTAGES OF PROPOSED SYSTEM	5
5	SYSTEM IMPLEMENTATION	6
5.1	SYSTEM ARCHITECTURE DIAGRAM	6
5.2	SYSTEM FLOW	7
5.3	LIST OF MODULES	8
5.4	MODULE DESCRIPTION	8
6	RESULTS AND DISCUSSION	10
7	APPENDIX	13
7.1	SAMPLE CODE	13
7.2	OUTPUT SCREENSHOTS	14
	REFERENCES	16

CHAPTER 1 — INTRODUCTION

1.1 INTRODUCTION

People share opinions about films, products, services, and experiences through online reviews. Manually reading and classifying large numbers of reviews can take considerable time. Sentiment analysis automates this task by predicting whether a text expresses a positive or negative opinion. Deep learning models can learn useful patterns from labelled examples, and recurrent neural networks are designed to process sequential data such as text.

1.2 PROBLEM STATEMENT

To develop a deep learning system that accepts a movie review and predicts whether its overall sentiment is positive or negative using a model trained on labelled review examples.

1.3 OBJECTIVES

- Load and inspect a labelled movie-review dataset.
- Convert reviews into integer sequences and pad or truncate them to a fixed length.
- Build and train a Long Short-Term Memory model for binary sentiment classification.
- Evaluate the model using training, validation, and test metrics.
- Prepare a prediction function for classifying new movie reviews.

1.4 SCOPE

The project focuses on binary classification of English-language movie reviews from the IMDB dataset. Performance on other domains, languages, sarcasm, and mixed sentiment may differ.

CHAPTER 2 — LITERATURE REVIEW

2.1 SENTIMENT ANALYSIS

Sentiment analysis identifies opinions and emotional polarity in text. Earlier approaches often used sentiment dictionaries or conventional machine learning with manually designed text features. Deep learning approaches learn text representations from examples and can model patterns in word sequences.

2.2 RECURRENT NEURAL NETWORKS AND LSTM

A recurrent neural network processes sequence elements in order while carrying a hidden state. Long Short-Term Memory networks use input, forget, and output gates together with a cell state to control information flow and help retain relevant information over longer sequences.

2.3 DATASET

This implementation uses the IMDB movie-review dataset distributed by Keras, containing 25,000 training reviews and 25,000 test reviews. The vocabulary is limited to the 10,000 most frequent words. Reviews are represented as integer sequences and labelled as negative or positive.

2.4 PROJECT APPROACH

The model combines an embedding layer, an LSTM layer, dropout regularization, and a sigmoid output layer. Validation performance is monitored during training, followed by evaluation on the separate test set.

CHAPTER 3 — SYSTEM REQUIREMENTS

3.1 HARDWARE REQUIREMENTS

- Laptop or desktop computer.
- Internet connection for Google Colab and dataset access.
- At least 4 GB RAM recommended.
- A graphics processing unit is optional for this model.

3.2 SOFTWARE REQUIREMENTS

- Python.
- Google Colab or Jupyter Notebook.
- TensorFlow and Keras.
- NumPy and Pandas.
- Matplotlib for plotting training curves.
- Streamlit, optional, for an interactive web interface.

3.3 LIBRARIES USED

- TensorFlow and Keras: building, training, and evaluating the neural network.
- NumPy: numerical operations.
- Pandas: inspecting dataset labels and information.
- Matplotlib: visualizing training curves and dataset information.
- Streamlit: optional interface for interactive predictions.

CHAPTER 4 — SYSTEM OVERVIEW

4.1 EXISTING SYSTEM

Manual review analysis requires a person to read each review and determine its sentiment. This process can be slow for large datasets and may lead to inconsistent decisions.

4.1.1 DRAWBACKS OF EXISTING SYSTEM

- Manual processing can be time-consuming.
- Results may vary between reviewers.
- Large collections of reviews are difficult to analyse quickly.
- Sarcasm, irony, and context can remain difficult for automated systems.

4.2 PROPOSED SYSTEM

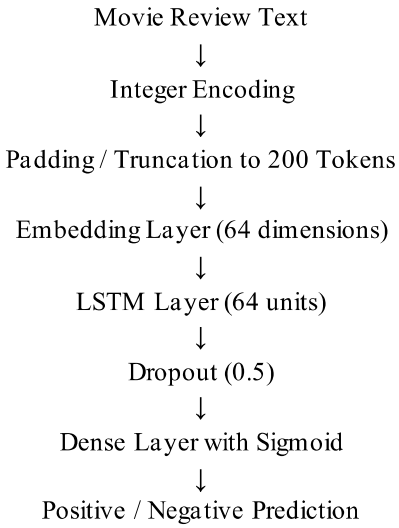
The proposed system uses an LSTM-based deep learning model to classify IMDB reviews as positive or negative. Text is represented as integer sequences, padded to a fixed length, processed by an embedding layer and LSTM layer, and classified by a sigmoid output unit.

4.2.1 ADVANTAGES OF PROPOSED SYSTEM

- Automates binary sentiment classification.
- Learns sequence patterns from labelled examples.
- Uses a publicly available dataset and common deep learning libraries.
- Can be extended with a user interface for live predictions.

CHAPTER 5 — SYSTEM IMPLEMENTATION

5.1 SYSTEM ARCHITECTURE DIAGRAM



5.2 SYSTEM FLOW

1. Load the IMDB dataset from Keras.
2. Inspect sentiment labels and review lengths.
3. Decode a sample review to inspect readable text.
4. Pad or truncate reviews to 200 tokens.
5. Create and compile the LSTM model.
6. Train for three epochs and monitor validation performance.
7. Evaluate the model on the separate test dataset.
8. Use a prediction function and optionally build a Streamlit interface.

5.3 LIST OF MODULES

- Dataset loading module
- Exploratory analysis module
- Text preprocessing module
- Model construction module
- Model training module
- Evaluation module
- Sentiment prediction module

5.4 MODULE DESCRIPTION

Dataset loading: Loads the IMDB dataset and separates training and test reviews.

Exploratory analysis: Inspects sentiment labels and example review sequences.

Text preprocessing: Pads and truncates reviews to a common sequence length of 200 tokens.

Model construction: Creates the embedding, LSTM, dropout, and dense layers.

Model training: Trains the network using the Adam optimizer and binary cross-entropy loss.

Evaluation: Uses accuracy and loss; additional classification metrics can be calculated after test evaluation.

Sentiment prediction: Accepts review text and returns a positive or negative prediction. Prediction-time encoding must match the training vocabulary.

CHAPTER 6 — RESULTS AND DISCUSSION

6.1 TRAINING RESULTS

The following values were recorded from the model training run.

Epoch	Training Accuracy	Validation Accuracy	Training Loss	Validation Loss
1	76.64%	82.80%	0.4808	0.4187
2	88.45%	86.14%	0.3018	0.3676
3	91.83%	86.58%	0.2260	0.3600

Training accuracy increased across the three epochs, while validation accuracy reached 86.58%. The difference between training and validation accuracy suggests some overfitting may be present. Test-set loss and accuracy have not yet been supplied and should be recorded after running model.evaluate on the test data.

6.2 EVALUATION METRICS

- Accuracy measures the proportion of correctly classified reviews.
- Precision measures the proportion of predicted positive reviews that are actually positive.
- Recall measures the proportion of actual positive reviews correctly identified.
- F1-score is the harmonic mean of precision and recall.
- A confusion matrix summarizes correct and incorrect predictions for each class.

6.3 LIMITATIONS

- The model performs binary sentiment classification only.
- Sarcasm, irony, and mixed opinions may be difficult to classify.
- Performance may decrease on text from domains different from movie reviews.
- The prediction function must use the same vocabulary conventions as training.
- Test metrics and application results should be added after execution.

CHAPTER 7 — APPENDIX

7.1 SAMPLE CODE

```
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Embedding, LSTM, Dropout, Dense

vocabulary_size = 10000
maximum_length = 200

(x_train, y_train), (x_test, y_test) = imdb.load_data(
    num_words=vocabulary_size
)

x_train_padded = pad_sequences(
    x_train, maxlen=maximum_length,
```

```
    padding="post", truncating="post"
)
x_test_padded = pad_sequences(
    x_test, maxlen=maximum_length,
    padding="post", truncating="post"
)

model = Sequential([
    Input(shape=(maximum_length,)),
    Embedding(vocabulary_size, 64, mask_zero=True),
    LSTM(64),
    Dropout(0.5),
    Dense(1, activation="sigmoid")
])

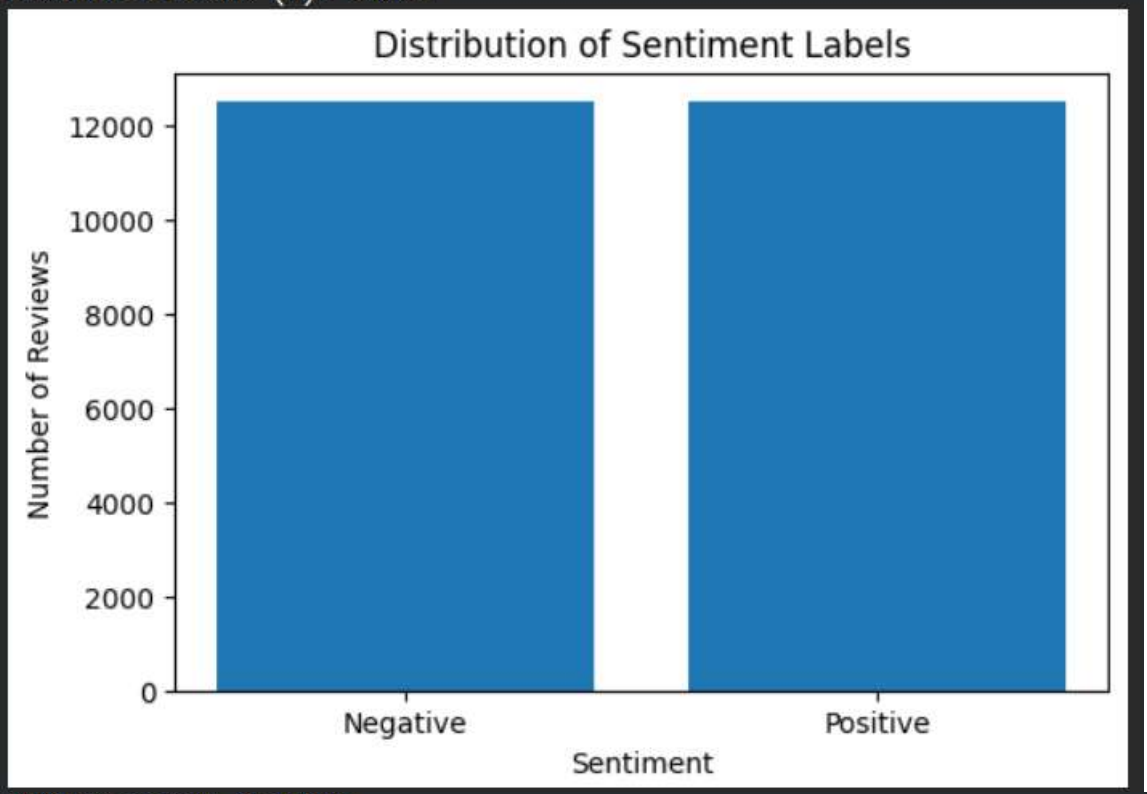
model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

history = model.fit(
    x_train_padded, y_train,
    epochs=3, batch_size=64,
    validation_split=0.2
)

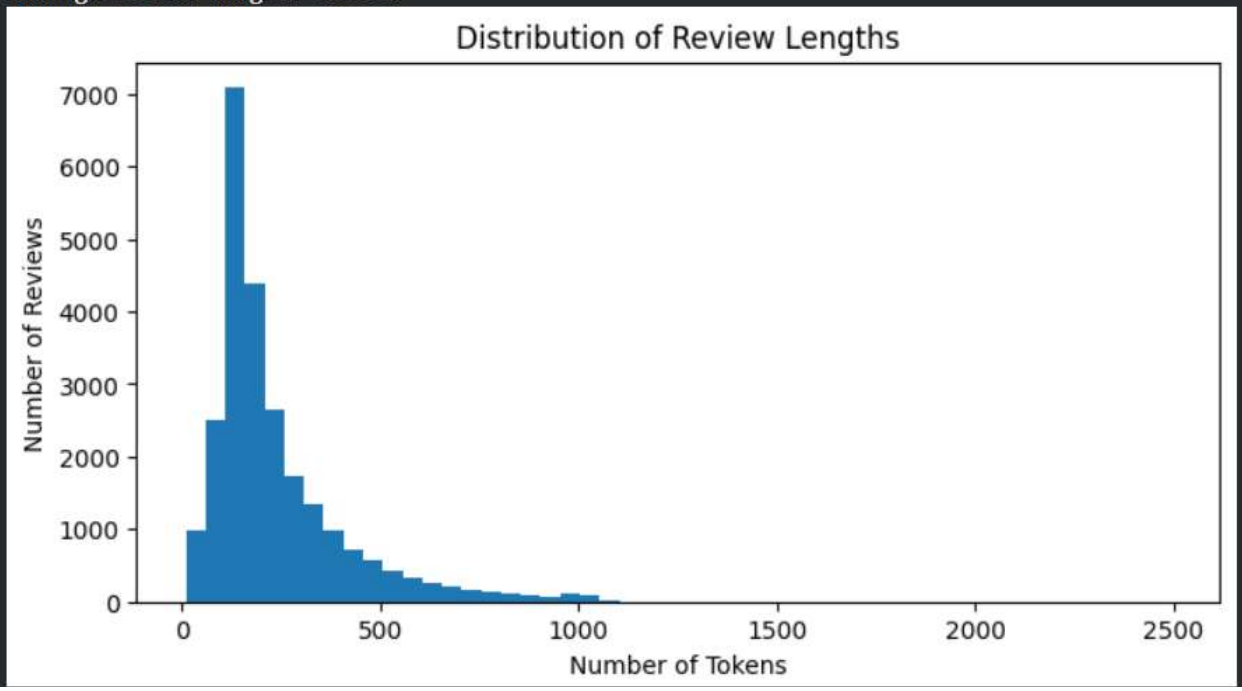
test_loss, test_accuracy = model.evaluate(
    x_test_padded, y_test
)
```

```
print("Test accuracy:", test_accuracy)
```

```
Sentiment distribution:
Negative reviews (0): 12500
Positive reviews (1): 12500
```




```
Shortest review: 11 tokens
Longest review: 2494 tokens
Average review length: 238.71
```



```
Original review length: 218
Processed review length: 200

Original review:
[1, 14, 22, 16, 43, 530, 973, 1622, 1385, 65, 458, 4468, 66, 3941, 4, 173, 36, 256, 5, 25, 100, 43, 838, 112, 50, 670, 2, 9, 35, 480, 284, 5, 150, 4, 172, 112, 167, 2, 336, 3

Processed review:
[   1   14   22   16   43  530  973 1622 1385   65  458 4468   66 3941
    4  173   36  256    5   25  100   43  838  112   50  670    2    9
   35  480  284    5  150    4  172  112  167    2  336  385   39    4
  172 4536 1111   17  546   38   13  447    4  192   50   16    6  147
 2025   19   14   22    4 1920 4613  469    4   22   71   87   12   16
   43  530   38   76   15   13 1247    4   22   17  515   17   12   16
  626   18    2    5   62  386   12    8  316    8  106    5    4 2223
 5244   16  480   66 3785   33    4  130   12   16   38  619    5   25
  124   51   36  135   48   25 1415   33    6   22   12  215   28   77
   52    5   14  407   16   82    2    8    4  107  117 5952   15  256
    4    2    7 3766    5  723   36   71   43  530  476   26  400  317
   46    7    4    2 1029   13  104   88    4  381   15  297   98   32
 2071   56   26  141    6  194 7486   18    4  226   22   21  134  476
   26  480    5  144   30 5535   18   51   36   28  224   92   25  104
    4  226   65   16]

Training data shape: (25000, 200)
Testing data shape: (25000, 200)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
lstm (LSTM)	?	0 (unbuilt)
dropout (Dropout)	?	0
dense (Dense)	?	0 (unbuilt)

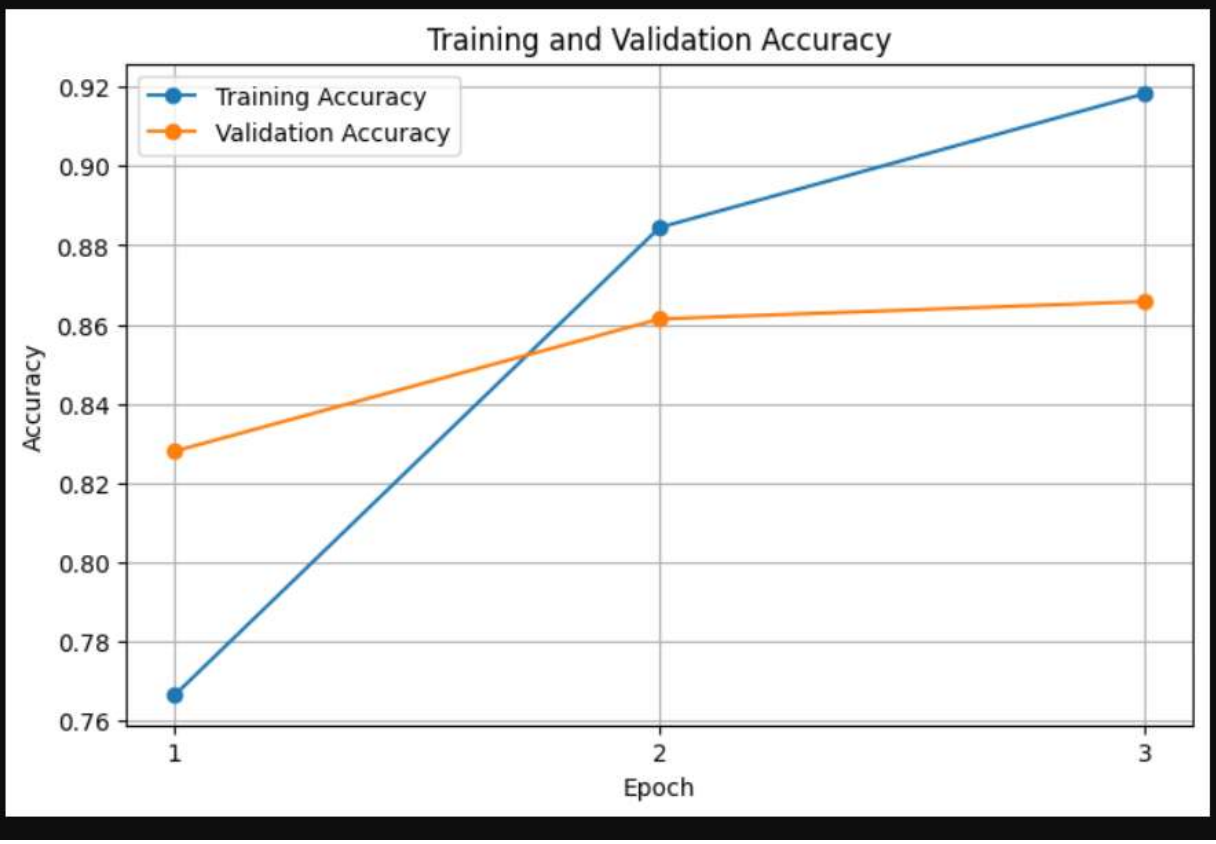
Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)

Model: "sequential_1"

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	(None, 200, 64)	640,000
lstm_1 (LSTM)	(None, 64)	33,024
dropout_1 (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 1)	65

Total params: 673,089 (2.57 MB)
Trainable params: 673,089 (2.57 MB)
Non-trainable params: 0 (0.00 B)

```
Review: This movie was absolutely fantastic and entertaining
Predicted sentiment: Positive
Positive sentiment score: 0.8405
```




```
Review: This movie was absolutely fantastic and entertaining
Predicted sentiment: Positive
Positive sentiment score: 0.8405
```


REFERENCES

9. TensorFlow Documentation. https://www.tensorflow.org/
10. Keras Documentation. https://keras.io/
11. TensorFlow Keras IMDB Dataset Documentation. https://www.tensorflow.org/api_docs/python/tf/keras/datasets/imdb
12. Matplotlib Documentation. https://matplotlib.org/stable/
13. Google Colaboratory. https://colab.research.google.com/